Option H — Pine → tradelab → Alpaca

Visual walkthrough & backtesting capability scorecard

Prepared 2026-04-24 · For Amit's review · Prerequisite: your approval before any code changes

> **The bottom line.** Option H keeps **TradingView as the single source of truth** for your strategies. Pine stays Pine. You never port a strategy to Python. Tradelab scores your strategies by importing the **trade list** TradingView produces; Alpaca executes them through a **small self-hosted webhook receiver** on your Windows box. You keep **~65%** of tradelab's backtesting features, lose automated parameter tuning & what-if tools, and eliminate the current two-copy architectural split entirely.

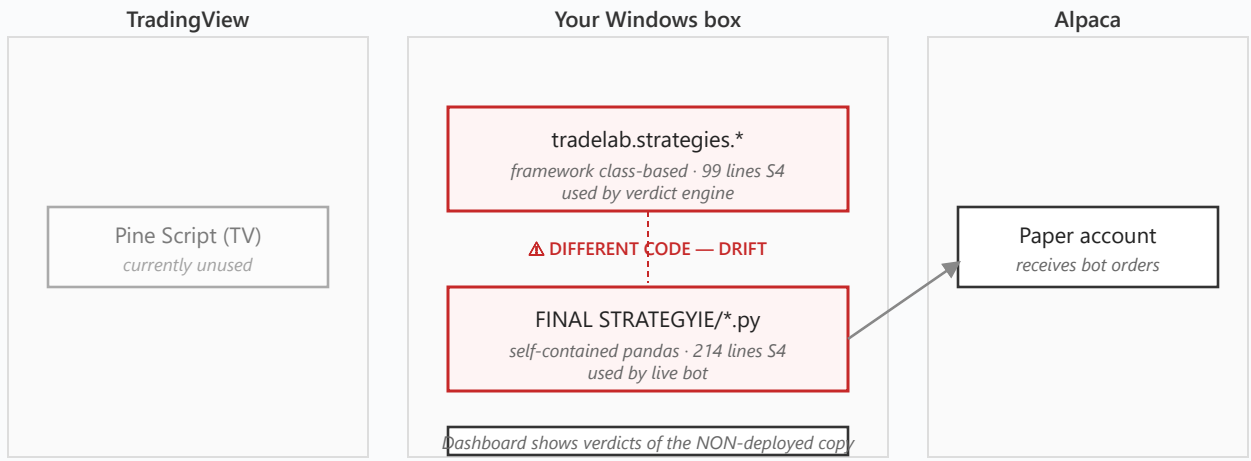
10	6	5	~1 wk
CAPABILITIES FULLY KEPT	PARTIAL (MANUAL WORKAROUND)	LOST (CODE-ONLY FEATURES)	ONE-TIME BUILD

WHAT THIS DOCUMENT COVERS

- Architecture today vs under Option H (before/after diagrams)
- End-to-end strategy lifecycle — 6 concrete steps
- Full capability scorecard — every tradelab backtest feature, kept / partial / lost
- Build cost, ongoing cost, per-strategy cost
- Risks & mitigations
- What you need to provide to kick off the build

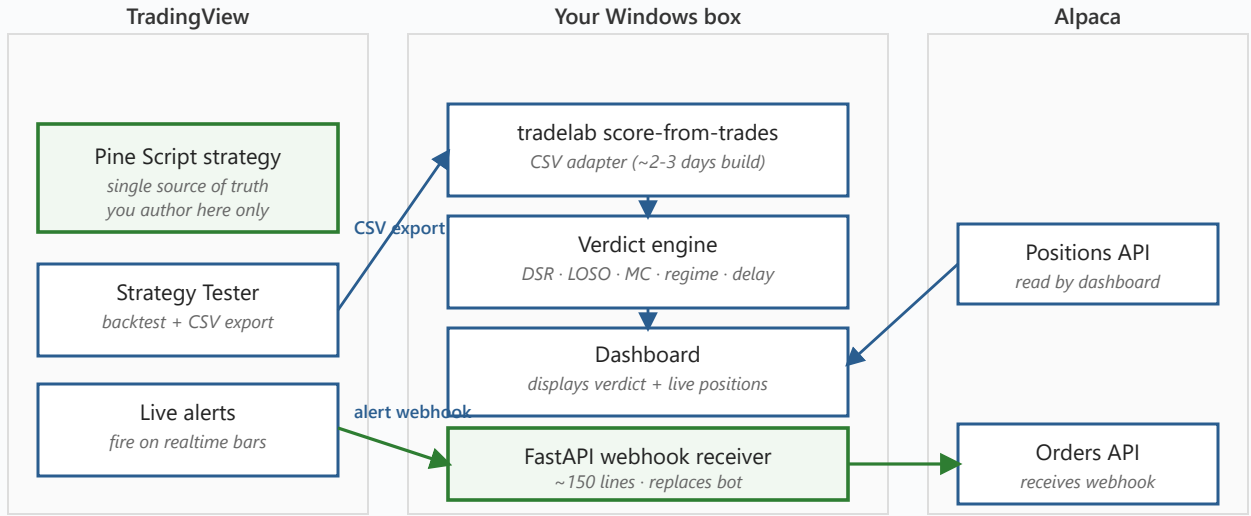
1. Architecture: before vs. after

TODAY (THE MECHANICAL GAP)



Today: two independent Python copies of each strategy, drifting from each other. Pine unused. Engine verdicts describe code that isn't trading.

UNDER OPTION H (SINGLE SOURCE OF TRUTH)



Option H: Pine is single source of truth. Blue = research/scoring flow (CSV). Green = live execution flow (alert webhooks). Zero Python translation.

2. Full strategy lifecycle — 6 steps

A concrete walkthrough of adding one new strategy from blank Pine editor to live paper trading. Approximate time budget next to each step.

1 Author in Pine Script

Where: TradingView web editor · Time: ~1h (varies by strategy complexity)

Write the strategy logic in Pine v6. Use `strategy.entry` / `strategy.exit` with descriptive `id` / `comment` strings — these travel to both the trade list CSV and the live alert payload. Add an `alert_message` JSON body that the webhook receiver can parse directly (symbol, side, quantity hint, strategy tag).

2 Backtest in TradingView Strategy Tester

Where: TradingView chart · Time: ~10 min per symbol/window

Load the Pine strategy on the target symbol. Set the date range. Click the "List of Trades" tab. Visually eyeball the trades and TradingView's built-in metrics (net P&L, PF, max DD, run-up). If the strategy is clearly broken, iterate in Pine before going further — no need to burn tradelab cycles on obvious losers.

3 Export trade list as CSV

Where: Strategy Tester · Time: 30 seconds

Click the export icon in the List of Trades tab, choose CSV. File saves to Downloads with every closed trade (entry/exit time + price, position size, signal tag, run-up, drawdown, P&L). This is your *ground truth* — exactly what TradingView saw.

4 Score in tradelab

Where: Windows terminal · Time: ~2 min

Run `tradelab score-from-trades my_strategy.csv --universe my_universe`. The CSV adapter converts TradingView's format to tradelab's internal trade-record schema, then runs the full robustness suite (DSR, LOSO, Monte Carlo, regime spread, etc.) on the ledger. Verdict: **ROBUST** / **INCONCLUSIVE** / **FRAGILE** plus a full signal breakdown.

5 Review verdict on dashboard

Where: http://localhost:8877 Research tab · Time: ~10 min

The new strategy appears as a row in the pipeline table. Heat colors, verdict pill, per-signal breakdown. If FRAGILE — iterate in Pine (Step 1), re-export (Step 3), re-score (Step 4). If ROBUST — proceed.

6 Deploy live (paper)

Where: TradingView alert dialog · Time: ~5 min

In TradingView, add an alert on the strategy's `strategy.entry` / `strategy.exit` events. Paste your webhook URL (your Windows box + ngrok / tunnel). Pine fires alerts on every live bar — the FastAPI receiver catches them, validates, and submits the order to Alpaca paper. Dashboard's Live Strategies card shows the position.

Per-strategy ongoing cost after step 6: roughly 15 min. When you change a Pine parameter, re-run Steps 3–5 (export → score → review). The Pine alert keeps running; no code is touched on the Windows side.

3. Backtesting capability scorecard

Every tradelab backtest feature, categorized by how Option H affects it.

Capability	Status	What it means under Option H
Core backtest (PF, WR, DD)	KEPT	TradingView Strategy Tester computes these directly. Tradelab re-derives from the trade list for consistency.
Deflated Sharpe Ratio (DSR)	KEPT	Works on trade-PnL vector alone. Full probability calculation.
Monte Carlo drawdown	KEPT	Shuffle-simulation on trade sequence. No code needed.
Regime-spread analysis	KEPT	Trades have timestamps; engine splits by regime.
Verdict aggregation (ROBUST / FRAGILE)	KEPT	Full rule engine runs on the signals it receives.
Strategy comparison (multi-strat)	KEPT	Compare two or more imported trade CSVs side-by-side.
Run history / audit DB	KEPT	Each CSV import logs a run; dashboard shows history.
Report generation (dashboard.html, quantstats)	KEPT	Works from trade data.
Live execution	KEPT	FastAPI webhook receiver replaces alpaca_trading_bot.py. Simpler, one file.
Live positions display	KEPT	Dashboard reads Alpaca API directly — unchanged.
Leave-One-Symbol-Out (LOSO)	PARTIAL	Needs per-symbol CSVs. Run the Pine strategy on each symbol separately in TV, export each, tradelab recombines. ~N times the export effort where N = symbol count.
Walk-forward validation	PARTIAL	Approximate by running Pine over non-overlapping rolling windows (e.g., 2020, 2021, 2022 separately). Manual but doable. No automated IS/OOS split.
Multi-symbol universe scoring	PARTIAL	Either run Pine per-symbol (trivial but N exports) or use `request.security` in Pine (but TV's CSV loses secondary-symbol context).

Capability	Status	What it means under Option H
Canary suite (rand / overfit / leak / survivor)	PARTIAL	Canaries are Python-native. They stay in tradelab but only test the engine itself, not Pine strategies.
Visual equity curve / QuantStats tearsheet	PARTIAL	Equity curve is derivable from the trade list. Tearsheet renders, though some drawdown-underwater plots need daily equity which TV doesn't export.
Param-landscape smoothness	PARTIAL	Requires running Pine at many parameter values. Manual grid-search in TV + batch import of each CSV. Painful but possible for small param grids.
Automated parameter optimization (Optuna)	LOST	Needs the code. No way to feed params to a TradingView strategy programmatically.
What-If parameter sliders	LOST	Same reason — needs a callable Python backtest.
Save-variant (clone & tune)	LOST	Pine strategies forked in TradingView, not tradelab. Manual rename + copy.
Entry-delay robustness stress test	LOST	Requires shifting entries programmatically. Can be approximated by manually editing Pine's entry bar logic and re-exporting, but it's tedious.
Noise-injection stress test	LOST	Requires injecting synthetic OHLC noise — Pine can't do that to itself.

SCORE SUMMARY

10 / 21	6 / 21	5 / 21
KEPT (48%)	PARTIAL (28%)	LOST (24%)

Translation: You keep the **entire verdict engine** and live execution. You lose the **parameter-tuning features** (Optuna, What-If, save-variant, entry-delay, noise). The partial items all have manual workarounds — time-consuming but not blocked. For a trader who tunes strategies visually in Pine anyway, the lost category is less important than it looks.

4. Cost & timeline summary

Line item	One-time	Ongoing
FastAPI webhook receiver (Python)	~1 day	~1h/month (TV IP changes, cert rotation)
Tradelab CSV adapter (`score-from-trades`)	~2-3 days	rare — only if TV changes CSV format
Runbook document (how to add a strategy)	~2h	—
Retire old bot + FINAL STRATEGYIE folder	~1h	—
Ngrok / Cloudflare Tunnel for webhook URL	~30 min setup	free tier or ~$8/month pro
TradingView account tier	—	Free tier works; Premium ($59.95/mo) only if you need Deep Backtest (>5k bars)
Per-strategy cost (write + test + deploy)	~1.5h first time	~15 min per parameter iteration

COMPARED TO ALTERNATIVES

- **vs. current dual-code architecture:** saves drift-maintenance forever. Eliminates the two-copy problem at the root.
- **vs. Option A (manual Python translation):** saves ~0.5–1 day of porting *every single time* a Pine strategy is created or modified.
- **vs. Option G (TradersPost SaaS at $99/mo):** saves $1,188/year, adds ~1 day build. Breakeven: ~10 days.

5. Risks & mitigations

R1 — TradingView changes CSV export format
Has been stable since ~2023. Still, TV ships UI changes frequently. *Mitigation:* CSV adapter includes a schema-version check that fails loudly; downgrade to "paste trade list into the dashboard" manual fallback while schema updates.

R2 — Webhook receiver downtime = missed trades
If the receiver crashes at 09:32, any alerts between then and restart are silently dropped. *Mitigation:* run as Windows service with auto-restart; a watchdog that compares "expected positions from last hour of alerts" vs "actual Alpaca positions" and alarms on divergence. TradersPost warns about the same failure mode; we at least control ours.

R3 — TradingView alert IP ranges change unannounced
TV has updated webhook source IPs twice in the last 18 months. Firewall rules fail silently. *Mitigation:* receiver validates a shared-secret in the alert JSON body, not IP. Works regardless of source IP.

R4 — Pine backtest ≠ live behavior (fill-model drift)
TV's Strategy Tester fills at bar close; live alerts fire in real time. Outcomes can differ. *Mitigation:* run first real strategy with tiny paper position sizes; compare live trade ledger vs backtest ledger over 2 weeks; flag if PF delta > 20%. Same risk exists under every option that uses TV — not unique to H.

R5 — Complex Pine features (multi-symbol, bracket orders) don't export cleanly
For the ~30% of strategies that use `request.security` or pyramiding, the CSV loses context. *Mitigation:* fall back to manual Python translation (Option A) for those specific strategies. Document which Pine features trigger the fallback in the runbook.

6. What you'll need to provide

- **One working Pine Script strategy** (even WIP or placeholder logic is fine) — lets me design the CSV adapter against a real export rather than guessing the schema.
- **Confirmation:** paper-only for now, or potentially live later? Affects whether we bake in extra safety rails.
- **Preferred webhook tunnel:** ngrok / Cloudflare Tunnel / direct-port-forward. I'd default to Cloudflare Tunnel (free, no dynamic DNS).
- **Green light:** your sign-off on the scope below.

7. Proposed next steps if you approve

1. **Session 1 (~1 day):** prototype FastAPI webhook receiver on your Windows box. Wire up ngrok/Cloudflare tunnel. Do end-to-end test: TV alert → receiver → Alpaca paper buy/sell. Prove the live path before spending time on the scoring side.
2. **Session 2 (~2-3 days):** build the `tradelab score-from-trades` CSV adapter. Score your first strategy and display its verdict on the dashboard.
3. **Session 3 (~half day):** retire old bot + FINAL STRATEGYIE. Simplify alpaca_config.json. Update dashboard docs.
4. **Session 4 (~2h):** write the runbook. Ship.

Total build effort: about one focused week spread across 4 sessions.

Sign-off

If this matches your intent, say "go" and we'll start Session 1 next. If anything in the scorecard, architecture, or risks above looks wrong or needs adjusting, push back first — this is the cheap moment to course-correct.

_____

Amit · date